Subject: ACS 547, Noise Control Applications - Module 3 Assignment

Date: March 30, 2025 (Submitted)

Problem 1 - Washing Machine Mount Design

The Matlab code for this problem is listed in Appendix 1.

Problem 1a

Equation calculates the force produced by the imbalanced 1kg mass at a 0.5 m radius from the center of the washing machine as a function of angular velocity,

$$F = m \cdot \omega^2 \cdot r \tag{1}$$

where m is 0.5 kg, ω is the angular velocity in $\frac{\text{radians}}{\text{s}}$, and r is 0.5 m.

Figure 1 shows the force produced by the 1 kg load as a function of angular velocity in RPM.

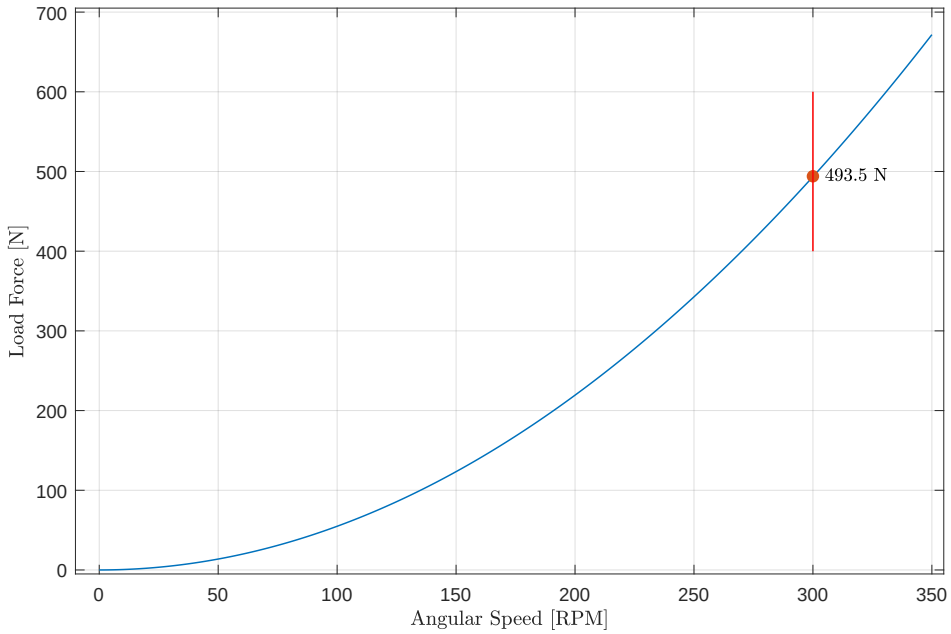

Figure 1: Load force versus washing machine angular speed.

At 300 RPM, or 5 Hz, the imbalanced load force amplitude is 493.5 N.

Problem 1b

The required spring stiffness, k_s, is 9,810 $\frac{\text{N}}{\text{m}}$. The mount's natural frequency, w_o, is 1.5 Hz.

Problem 1c

Figure 2 shows the transmission loss of the system for a set of damping ratios ranging from 0.001 to 1.

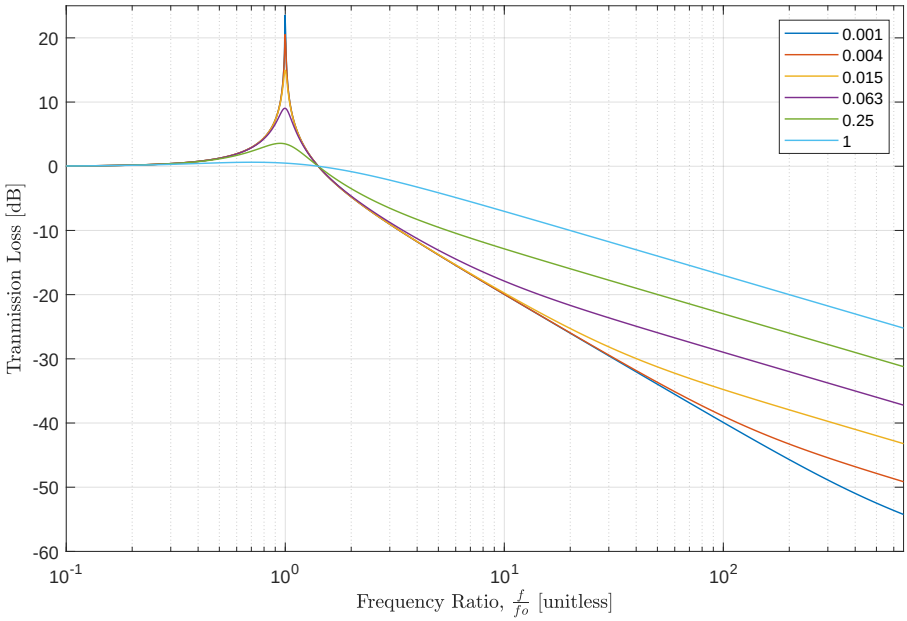

Figure 2: Transmission loss as a function of damping ratio. The natural frequency is 1.5 Hz.

Problem 1d

Figure 3 shows the force applied to the foundation.

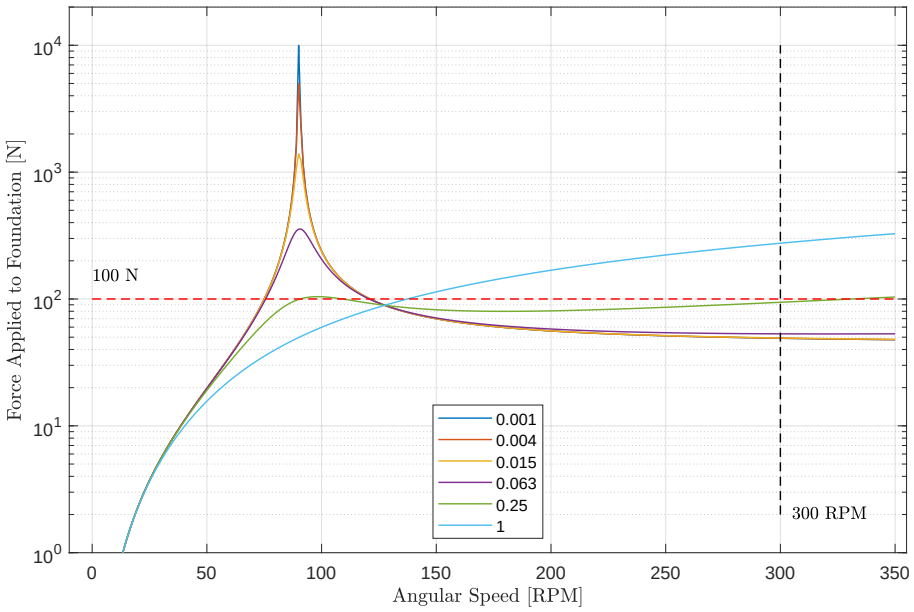

Figure 3: Force applied to the foundation as a function of damping ratio.

No damping ratio makes the applied force less than 100 N across the 0 to 350 RPM operating range.

The best damping ratio is $\zeta = 0.25$. The force applied to the foundation still exceeds the 100 N target in only two small RPM regions: 91 to 109, and 331 to 350 RPM. In both of these regions the force is approximately 104 N.

Problem 1e

The viscous damping coefficient is 519.4 $\frac{\text{kg}}{\text{s}}$ based on a total mass of 110 kg.

Figure 4 shows the admittance.

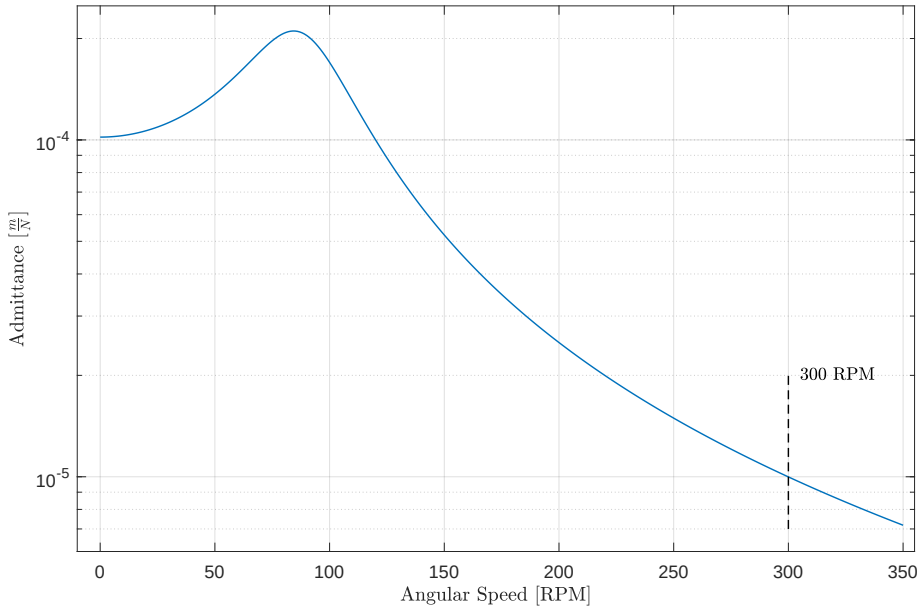

Figure 4: Admittance versus RPM.

The admittance at 300 RPM is 1e-5 $\frac{\text{m}}{\text{N}}$.

Problem 1f

Figure 5 shows the displacement of the washing machine.

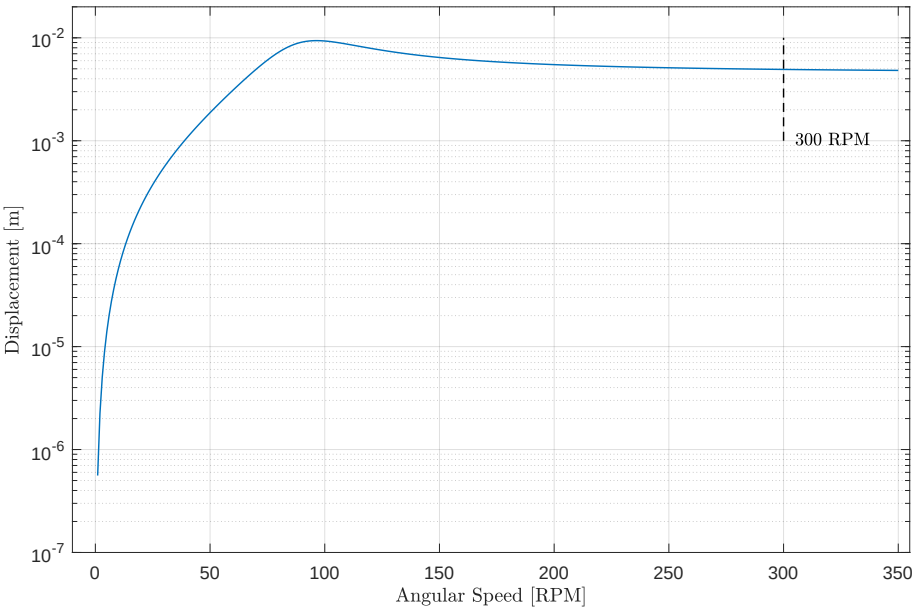

Figure 5: Displacement versus RPM.

The displacement of the washing machine at 300 RPM is approximately 4.9×10^{-3} m. The washing machine undergoes larger momentary displacements as is spins up to 300 RPM, in particular around 100 RPM.

Problem 2 - Washing Machine Mount Design (2 DOF)

The Matlab code for this problem is listed in Appendix 2.

Problem 2a

Figure 1 illustrates the two-stage vibration mount showing the washing machine and the "raft".

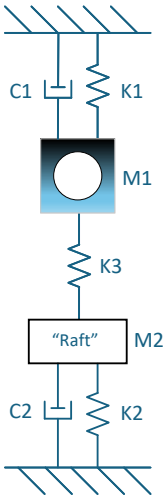

Figure 1: Two-stage vibration mount system for the washing machine.

Problem 2b

Table 1 lists the initial stiffness and damping coefficients for the system.

M1	110	kg
K1	100	$\frac{\text{N}}{\text{m}}$
C1	5	$\frac{\text{kg}}{\text{s}}$ [1]

M2	100	kg
K2	100	$\frac{\text{N}}{\text{m}}$
C2	5	$\frac{\text{kg}}{\text{s}}$ [2]

M3	0	kg
K3	1,000	$\frac{\text{N}}{\text{m}}$
C3	0	$\frac{\text{kg}}{\text{s}}$ [3]

Table 1: Initial values of stiffness and damping coefficients.

Damping was implemented using the damping vector[4].

[1] Element of damping vector.
[2] Element of damping vector.
[3] Element of damping vector.
[4] Not equivalent to using a complex-valued stiffness vector; the imaginary part is the damping for the respective spring

Problem 2c

Figure 2 shows the admittance of the washing machine and the raft.

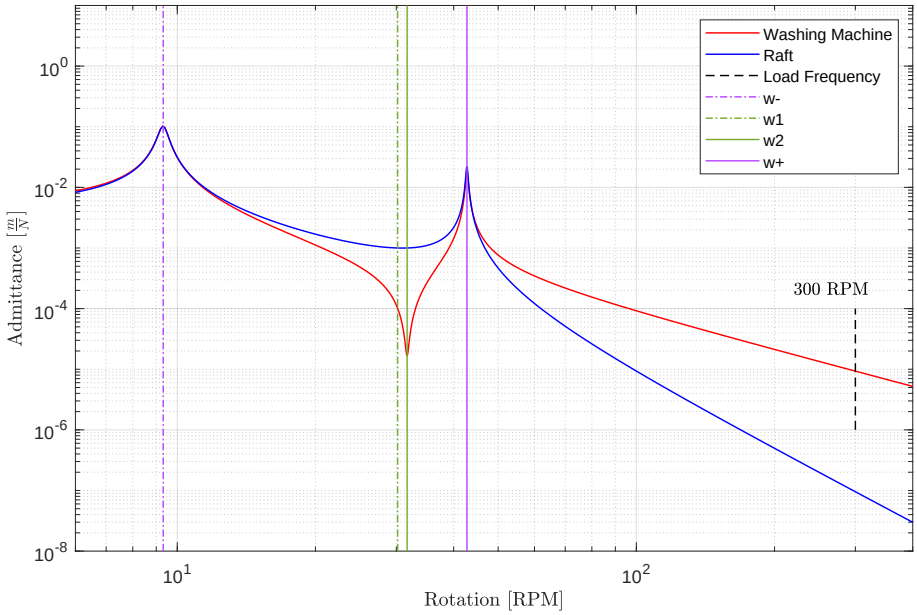

Figure 2: Admittance of the washing machine and the raft with a force on the washing machine.

The admittance of the washing machine is 9.31×10^{-6} $\frac{\text{m}}{\text{N}}$ at 300 RPM.

Problem 2d

Figure 3 show the displacement of the washing machine and the raft.

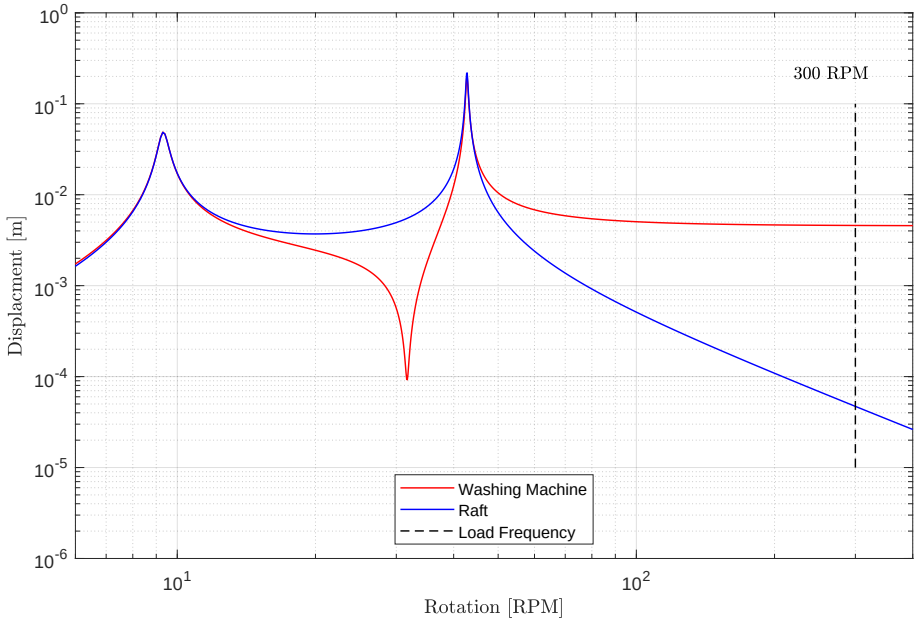

Figure 3: Displacement of the washing machine and the raft with a force on the washing machine.

At 300 RPM, the washing machine displacement is 4.6×10^{-3} m, less than 5.0×10^{-3} m for the first-order system. The displacement of the raft is 4.7×10^{-5} m.

Problem 2e

Figure 4 shows the force applied to the ground by the raft.

The force is is calculated by multiplying the displacement of the raft by the K2 spring constant.

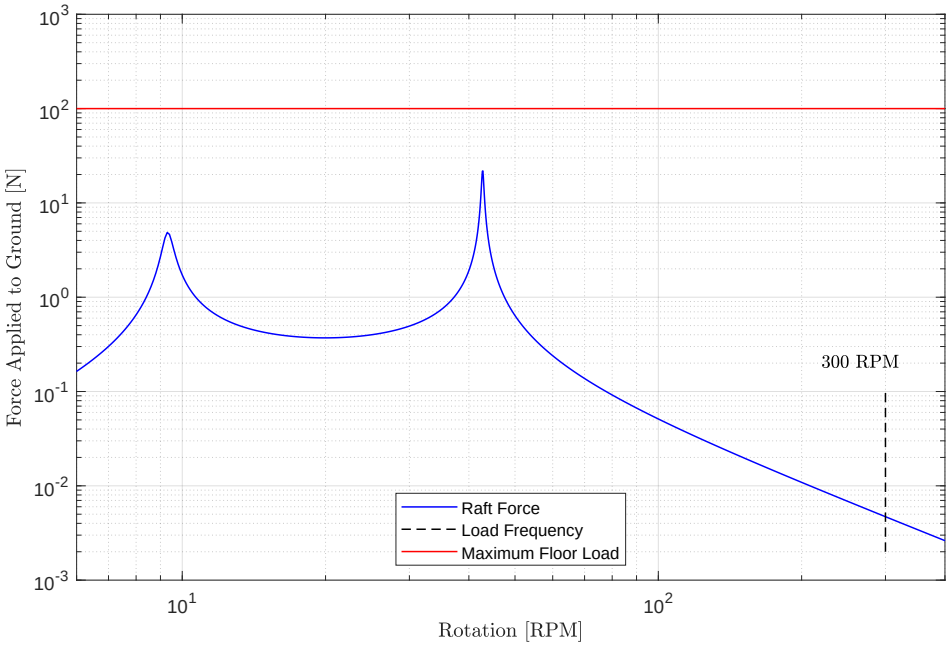

Figure 4: Force applied to the ground by the raft.

The ground force at 300 RPM is 4.7×10^{-3} N. The force on the ground is always less than 100 N.

Problem 2f

Table 1 shows

The initial parameter set chosen, see Table **??**, facilitated a displacement of the washing machine of 4.6×10^{-3} m at 300 RPM, which is less than the 5.0×10^{-3} m found for the 1 DOF system. The force applied to the ground by the raft is less than the required 100 N maximum.

Problem 2g

The initial parameter set chosen, see Table **??**, facilitated a displacement of the washing machine of 4.6×10^{-3} m at 300 RPM, which is less than the 5.0×10^{-3} m found for the 1 DOF system. The force applied to the ground by the raft is less than the required 100 N maximum.

Problem 3 - Washing Machine Dynamic Vibration Absorber Design

The Matlab code for this problem is listed in Appendix 3.

Part 3a

Figure 1 illustrates the dynamic vibration absorber (DVA) system.

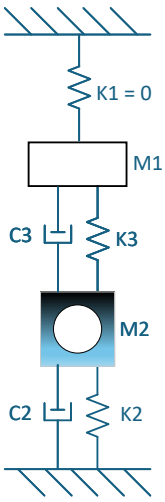

Figure 1: Dynamic vibration absorber system for the washing machine.

Table 2 lists the stiffness and damping coefficients for the system.

M1	15	kg
K1	0	$\frac{N}{m}$
C1	0	$\frac{kg}{s}$

M2	110	kg
K2	9,810	$\frac{N}{m}$
C2	1,962	$\frac{kg}{s}$ [5]

M3	0	kg
K3	1e5	$\frac{N}{m}$
C3	1e4	$\frac{kg}{s}$ [6]

Table 2: Stiffness and damping coefficients for the DVA system.

Damping was implemented by adding with an imaginary component to the respective spring stiffness[7].

[5] 0.2 fraction of respective spring stiffness.
[6] 0.1 fraction of respective spring stiffness.
[7] Not equivalent to using a damping vector in the argument to the given Matlab nDOF function script-file.

Part 3b

Figure 2 shows the admittance response of the original system and DVA system based on the equations presented in the tutorial lecture on DVA system analysis.

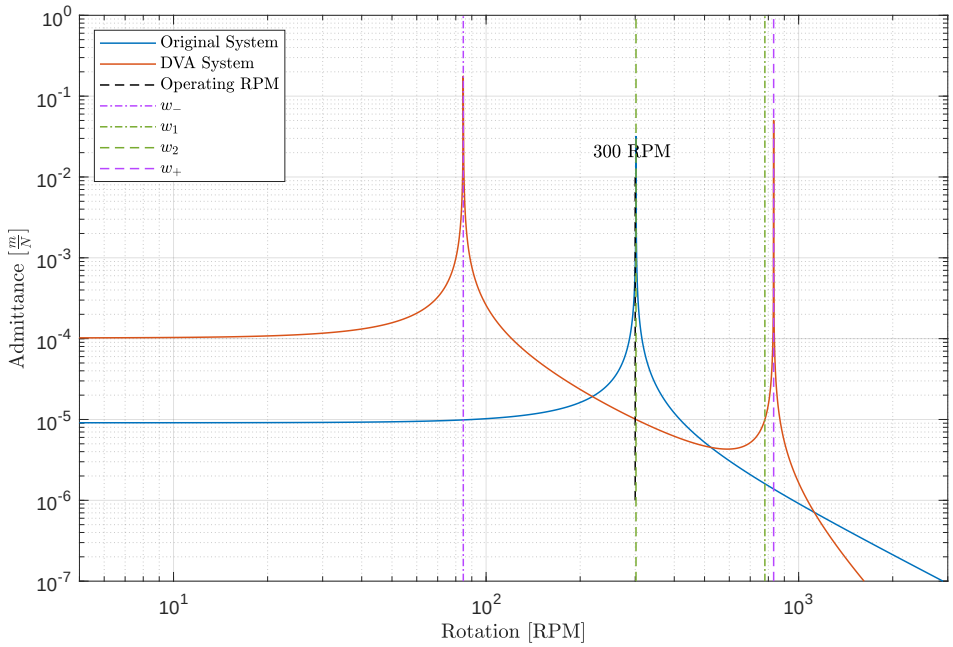

Figure 2: Admittance responses based on the equations from the DVA tutorial lecture.

The washing machine admittance at 300 RPM is reduced from the blue curve resonance peak to a point between the two resonances on the red curve).

Part 3c - Admittance

Figure 3 shows the admittance response of the DVA system using the nDOF function script-file. The stiffness of the coupling spring, k_3, is set to place the null of the admittance response at the 300 RPM operating state of the washing machine. *The admittance at 300 RPM is approximately* 1.2×10^{-6} $\frac{\text{m}}{\text{N}}$.

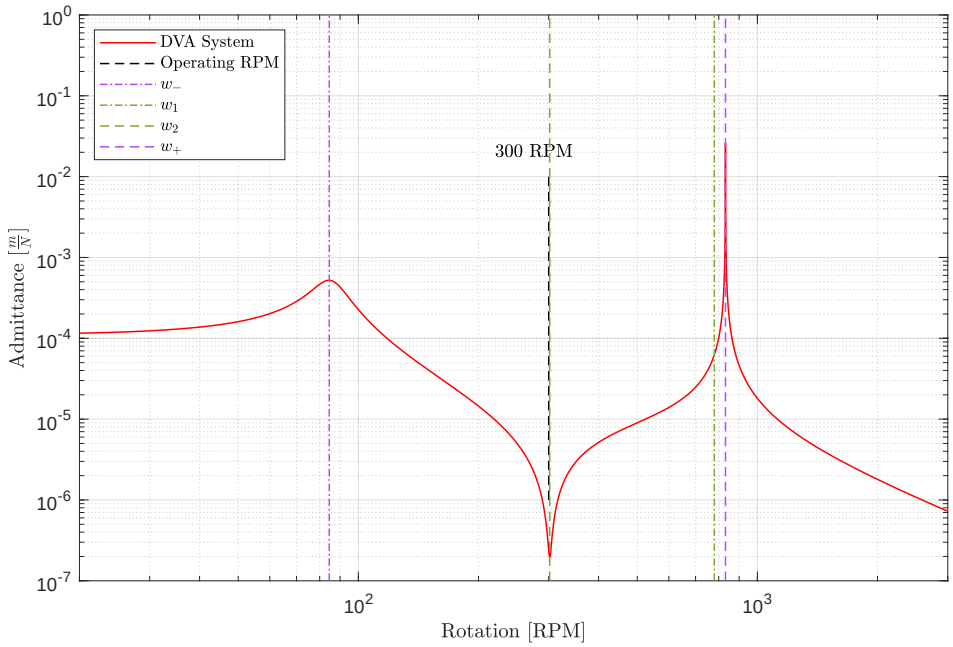

Figure 3: Solution using nDOF script-file function.

Part 3d - Displacement

Figure 4 shows the displacement of the washing machine as a function of RPM. The ramp up to 300 RPM occurs quickly so that the large displacements are transient and the maximum speed is 350 RPM (the large displacement associated with the resonance peak at higher frequencies is not reached). *The displacement at 300 RPM is approximately* 38.2×10^{-6} m.

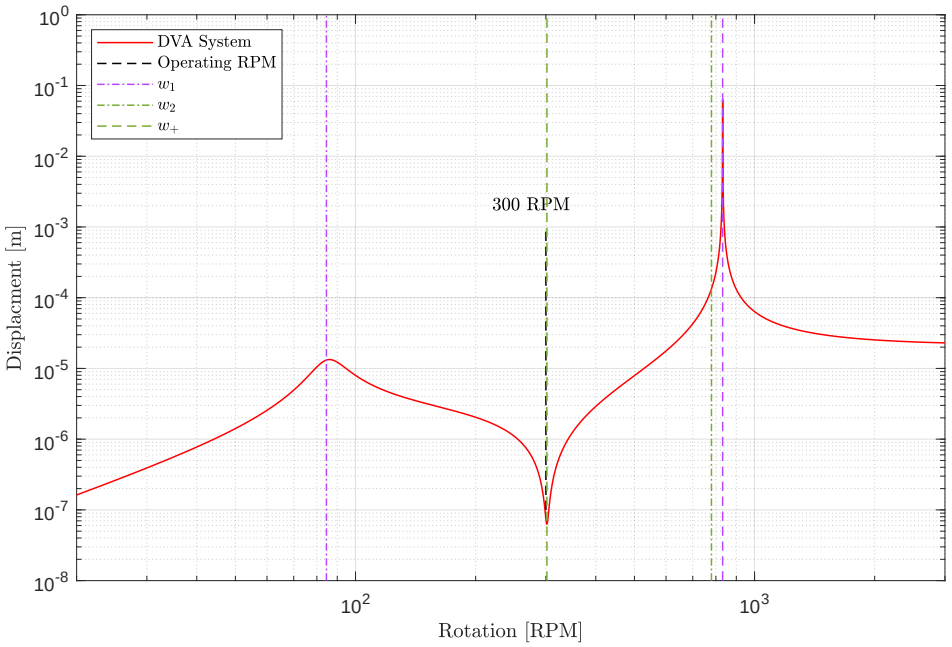

Figure 4: Displacement of the washing machine.

Part 3e

Figure 5 shows the force applied to the floor by the washing machine. *The force applied to the floor at 300 RPM is approximately* 3.75×10^{-3} m.

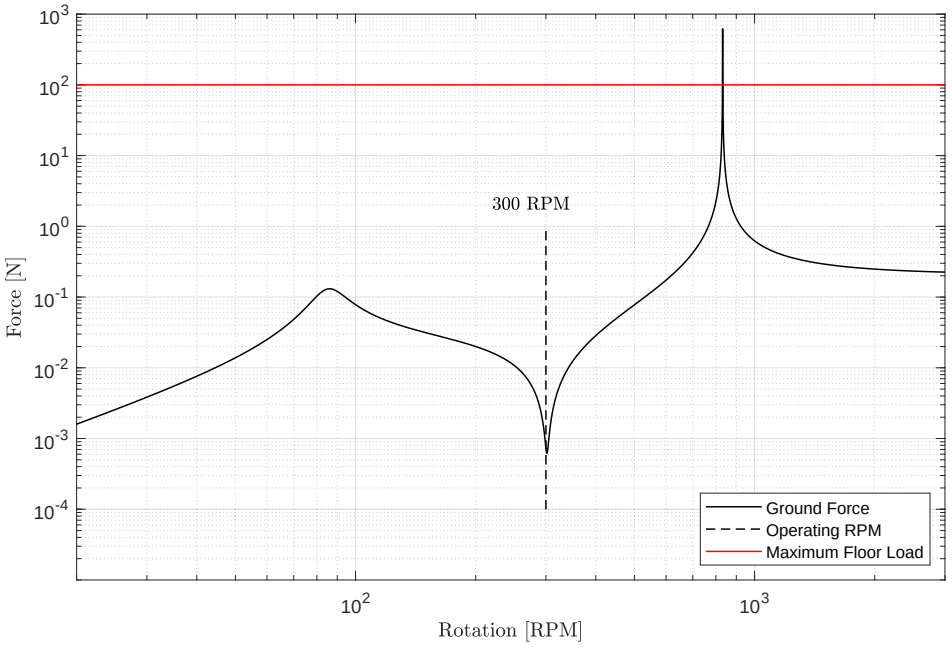

Figure 5: Force applied to the floor.

Problem 4 - Washing Machine Abuse Testing

The Matlab code for this problem is listed in Appendix 4.

Part 4a

Summary

ph

1 Appendix - Matlab Code for Problem 1

```
1
2
3
4  %% Synopsis
5
6  % Problem 1 - Washing Machine Mount Design - 1 Degree of Freedom (DOF)
7
8
9
10 %% Environment
11
12 close all; clear; clc;
13 % restoredefaultpath;
14
15 addpath( genpath( '../00 Support' ), '-begin' );
16
17 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
18 set( 0, 'DefaultFigureWindowStyle', 'normal' );
19 set( 0, 'DefaultLineLineWidth', 0.8 );
20 set( 0, 'DefaultTextInterpreter', 'Latex' );
21
22 format LongG;
23
24 pause( 1 );
25
26
27
28 %% Anonymous Function Definitions
29
30 h_force = @( force_mass, rotation_speed, load_distance  )  force_mass .* rotation_speed.^2 .*
       load_distance;
31
32 h_transmissibility = @( epsilon, r )  sqrt( ( 1 + (2.*epsilon.*r).^2 ) ./ ( ( 1 - r.^2 ).^2 + (
       2.*epsilon.*r ).^2 ) );
33
34 rpm_conversion_to_radians_per_second = 0.10472;
35
36
37
38 %% Washing Machine Information
39
40 machine.mass = 100;  % kg
41 machine.rotation_speed = 31.4;  % radians\s or 300 rotations per minute (RPM)
42 machine.load_mass = 10;  % kg
43 machine.static_displacement = 1e-2;  % m
44
45
46
47 %% Load Force Information
48
49 dynamic_force.mass = 1;  % kg
50 dynamic_force.radius = 0.5;  % m
51
52 force_frequency = 300 * rpm_conversion_to_radians_per_second / ( 2 * pi );  % 5 Hz
53
54 % Maximum force applied to foundation is 100 N.
55
56
57
58 %% Part 1a
59
60 rpm = 0:1:350;
61     angular_velocity = rpm .* rpm_conversion_to_radians_per_second;  % radians\s
62
63
64 figure( 'Name', 'Problem 1a - Load Force Versus Angular Velocity' ); ...
65     plot( rpm, h_force( dynamic_force.mass, angular_velocity, dynamic_force.radius ) );  hold on;
66     plot( 300, 494, 'Marker', '.', 'MarkerSize', 20 );
67     line( [ 300, 300 ], [ 400 600 ], 'Color', 'r' );
68     text( 305, 494, '493.5 N' );  grid on;
69     xlabel( 'Angular Speed [RPM]' );  ylabel( 'Load Force [N]' );
70     axis( [ -10 355  -5 705 ] );
71
72
73
```

```
%% Part 1b

maximum_allowable_displacement = 1e-2;  % m
    ks = ( machine.load_mass * 9.81) / maximum_allowable_displacement;  % 9,810 N\m


total_mass = machine.mass + machine.load_mass;  % 110 kg
    wo = sqrt( ks / total_mass );  % 9.4 radians\s
        fo = wo / (2*pi);  % 1.5 Hz - Natural frequency of the mount.



%% Part 1c

frequency_set = 0.1:0.01:1e3;
    r = frequency_set ./ fo;

damping_ratio = logspace( log10(0.001), log10(1), 6 );


figure( 'Name', 'Problem 1c - Transmission Loss' ); ...
    hold on;
    %
    for index = 1:1:numel( damping_ratio )
        plot( r, 10*log10( h_transmissibility( damping_ratio( index ), r ) ) );
    end
    %
    xlabel( 'Frequency Ratio, $\frac{f}{fo}$ [unitless]' );  ylabel( 'Tranmission Loss [dB]' );
        legend( '0.001', '0.004', '0.015', '0.063', '0.25', '1', 'Location', 'NorthEast' );
    axis( [ 0.1 665  -60 25 ] );
    grid on;  box on;
    set( gca, 'XScale', 'log' );



%% Part 1d

rpm = 0:1:350;  % rotations per minute
    rpm_conversion_to_radians_per_second = 0.10472;  % radians\s
        angular_velocity = rpm .* rpm_conversion_to_radians_per_second;  % radians\s

frequency_set = angular_velocity / (2*pi);
    r = frequency_set ./ fo;


figure( 'Name', 'Problem 1d - Applied Force to Foundation' ); ...
    hold on;
    %
    for index = 1:1:numel( damping_ratio )
        plot( rpm, h_transmissibility( damping_ratio( index ), r ) .* h_force( dynamic_force.mass
            , angular_velocity, dynamic_force.radius ) );
    end
    %
    line( [ 300, 300 ], [ 2 1e4 ], 'Color', 'k', 'LineStyle', '--' );
        text( 305, 2, '300 RPM' );  grid on;
    line( [ 0 350 ], [ 100 100 ], 'Color', 'r', 'LineStyle', '--' );
        text( 0, 150, '100 N' );  grid on;
    xlabel( 'Angular Speed [RPM]' );  ylabel( 'Force Applied to Foundation [N]' );
        legend( '0.001', '0.004', '0.015', '0.063', '0.25', '1', 'Location', 'South' );
    axis( [ -10 355  1 2e4 ] );
    grid on;  box on;
    set( gca, 'YScale', 'log' );



%% Part 1e

% The best damping ratio is 0.25.

C = 0.25 * 2*sqrt( ks * total_mass );  % 519.4 kg\s

% The units of a viscous damping coefficient are Newton-seconds per meter (Ns/m) or kilograms per
    second (kg/s).


m = total_mass;
k = [ ks  0 ];
dampings = [ C  0 ];
```

```
    admittance = nDOF_direct_solution( m, k, dampings, frequency_set, 'admittance' );


figure( 'Name', 'Problem 1e - Admittance' ); ...
    semilogy( rpm, abs( admittance ) );  grid on;
    xlabel( 'Angular Speed [RPM]' );  ylabel( 'Admittance [$\frac{m}{N}$]' );
    set( gca, 'YScale', 'log' );
    %
    line( [ 300, 300 ], [ 7e-6 2e-5 ], 'Color', 'k', 'LineStyle', '--' );
        text( 305, 2e-5, '300 RPM' );  grid on;
    axis( [ -10 355  6e-6 2.5e-4 ] );


% temp2 = abs( admittance );
%     round( temp2( 301 ), 3, 'significant' )



%% Part 1f

displacement = abs( admittance ) .* h_force( dynamic_force.mass, angular_velocity, dynamic_force.
    radius ).';


figure( 'Name', 'Displacement' ); ...
    plot( rpm, displacement );  grid on;
    xlabel( 'Angular Speed [RPM]' );  ylabel( 'Displacement [m]' );
    line( [ 300, 300 ], [ 1e-3 1e-2 ], 'Color', 'k', 'LineStyle', '--' );
        text( 305, 1e-3, '300 RPM' );  grid on;
    set( gca, 'YScale', 'log' );
    axis( [ -10 355  1e-7 2e-2 ] );


temp2 = abs( displacement );
    round( temp2( 301 ), 3, 'significant' )



%% Clean-up

if ( ~isempty( findobj( 'Type', 'figure' ) ) )
    monitors = get( 0, 'MonitorPositions' );
        if ( size( monitors, 1 ) == 1 )
            autoArrangeFigures( 3, 4, 1 );
        elseif ( 1 < size( monitors, 1 ) )
            autoArrangeFigures( 2, 2, 2 );
        end
end

fprintf( 1, '\n\n\n*** Processing Complete ***\n\n\n' );



%% Reference(s)
```

2 Appendix - Matlab Code for Problem 2

```
1
2
3
4  %% Synopsis
5
6  % Problem 2 - Washing Machine Two-stage Mount Design
7
8
9
10 %% Note(s)
11
12 % A 2 degree-of-freedom (DOF) system will provide isolation performance 1 DOF system.
13
14 % Allowing the top and bottom masses, the washing machine and the raft,
15 % respectively, to each be attached to ground independently.
16
17 % With the force applied to the washing machine and the displacement of the
18 % washing machine, use FSF( :, 1, 1 ).  However, if the force is applied elsewhere,
19 % than FSF( :, 2, 2, ) must be used.
20
21
22
23 %% Environment
24
25 close all; clear; clc;
26 % restoredefaultpath;
27
28 addpath( genpath( './00 Support' ), '-begin' );
29
30 set( groot, 'DefaultFigurePosition', [ 100  750    750  500 ] ); % x, y, width, height
31
32 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
33 set( 0, 'DefaultFigureWindowStyle', 'normal' );
34 set( 0, 'DefaultLineLineWidth', 0.8 );
35 set( 0, 'DefaultTextInterpreter', 'Latex' );
36
37 format ShortG;
38
39 pause( 1 );
40
41
42
43 %% Define Anonymous Functions
44
45 h_f_to_rpm = @( f )  ( f*2*pi ) / 0.10471;
46
47 h_w_to_rpm = @( w )  h_f_to_rpm( w*2*pi);
48 h_w_to_f = @( w )  w/(2*pi);
49
50 h_rpm_to_f = @( rpm )  ( rpm * 0.10471 ) / ( 2 * pi );
51
52
53 h_force = @( force_mass, rotation_speed, load_distance  )  force_mass .* rotation_speed.^2 .*
       load_distance;
54
55
56
57 %% Problem 2a
58
59 % See report.
60
61
62
63 %% Problem 2b - Blocked Frequencies of Resonance
64
65 % The frequency of the forcing function is 5 Hz and the amplitude is 493.5 N.
66 fo = 5;  % Hz
67     wo = 2*pi*fo;  % 31.4 radians\s
68
69
70 m1 = 100 + 10;  % kg - Total mass of the washing machine and the load.
71     k1 = 1e2;  % N\m
72     c1 = 5;
73
74 m2 = 100;  % kg - UPPER LIMIT OF 100 kg
```

```
75      k2 = 1e2;  % N\m
76      c2 = 5;
77
78  k3 = 1e3;  % N\m -> Larger values produce stronger coupling (higher w+).
79
80
81  w1 = sqrt( ( k1 + k3 ) / m1 );  % 3.2 radians\s;  washing machine
82      f1 = w1 / (2*pi);  % 0.5 Hz
83          rpm1 = f1*(2*pi)/0.10471;  % 30.1 RPM
84
85  w2 = sqrt( ( k2 + k3 ) / m2 );  % 3.3 radians\s;  raft
86      f2 = w2 / (2*pi);  % 0.53 Hz
87          rpm2 = f2*(2*pi)/0.10471;  % 31.7 RPM
88
89  % Note:  If k3 is set to zero, then these frequencies represent the
90  % resonance frequencies for each mass on their own.
91
92
93
94  %% Problem 2b - Coupled Frequencies
95
96  mu_4 = ( k3^2 / (m1*m2) );  % 90.9 (radians\s)^4
97      mu = ( mu_4 )^0.25;  % 3.1 radians\s
98
99  w(1) = 0.5*( ( w1^2 + w2^2 )  +  sqrt( ( w1^2 - w2^2 )^2 + 4*mu_4 ) );
100 w(2) = 0.5*( ( w1^2 + w2^2 )  -  sqrt( ( w1^2 - w2^2 )^2 + 4*mu_4 ) );
101     w = sqrt( w );
102
103
104 w_minus = w(2);  %  0.97 radians\s  (lower than w1, 1, and w2, 1.05 radians\s)
105     f_minus = w(2)/(2*pi);  %  0.1545 Hz
106         rpm_minus = h_f_to_rpm( f_minus );  % 9.3 RPM
107 %
108 w_plus = w(1);  %  4.5 radians\s  (higher than w1, 1, and w2, 1.05 radians\s)
109     f_plus = w(1)/(2*pi);  %  0.71 Hz
110         rpm_plus = h_f_to_rpm( f_plus );  % 42.8 RPM
111 %
112 % Note(s):
113 %
114 %   The blocked frequencies, w1 and w2, are always between these two frequencies.
115
116 clear w mu_4;
117
118
119
120 %% Problem 2b - Mode Shapes
121
122 phi_plus = [ ...
123     1; ...
124     -(1/mu^2)*sqrt(m1/m2)*(w_plus^2 - w1^2), ...
125     ];
126 %
127 %   1
128 %   -1.10
129
130 phi_minus = [ ...
131     1; ...
132     (1/mu^2)*sqrt(m1/m2)*(w1^2 - w_minus^2), ...
133     ];
134 %
135 %   1
136 %   0.99
137
138
139 % Note(s):
140 %
141 % When m1 = m2 and k1 = k3,
142 %
143 %   a.)  At w_minus, m1 and m2 move the same amount and are in-phase.
144 %   b.)  At w_plus, m1 and m2 move the same amount, but are out-of-phase.
145
146 % In general, for a 2 DOF system there will be 2 modes (in-phase and out-of-phase).
147
148 % With different initial conditions (no forcing), the behaviour is a weighted sum of the two
        modes.
149
150
151
```

```
%% Problem 2c

masses = [ m1 m2 ];
stiffnesses = [ k1  k3  k2 ];
dampings = [ c1  0  c2 ];


rpm = 0:0.1:500;  % rotations per minute
    rpm_conversion_to_radians_per_second = 0.10472;
        angular_velocity = rpm .* rpm_conversion_to_radians_per_second;  % radians\s
            frequencies = angular_velocity / (2*pi);  % Hz

FRF = nDOF_direct_solution( masses, stiffnesses, dampings, frequencies, 'admittance' );


admittance = zeros( numel( frequencies ), 1 );

for index = 1:1:numel( frequencies )
    temp = diag( squeeze( FRF( index, 1, 1 ) ) );  % Check indexing to ensure force on washing
        machine.
        admittance( index ) = abs( temp );
end
%
clear temp temp2;

washing_machine.admittance = admittance;


admittance = zeros( numel( frequencies ), 1 );

for index = 1:1:numel( frequencies )
    temp = diag( squeeze( FRF( index, 1, 2 ) ) );  % Check indexing to ensure force on washing
        machine.
        admittance( index ) = abs( temp );
end
%
clear temp temp2;

raft.admittance = admittance;


figure( 'Name', 'Admittance' ); ...
    h1 = loglog( rpm, washing_machine.admittance, 'Color', 'r', 'LineWidth', 0.8 );  hold on;
    h2 = loglog( rpm, raft.admittance, 'Color', 'b', 'LineWidth', 0.8 );
    h3 = line( [ 300 300 ], [ 1e-6 1e-4 ], 'Color', 'k', 'LineStyle', '--' );  grid on;
        text( 220, 2e-4, '300 RPM' );
    %
    h4 = line( [ rpm_minus rpm_minus], [ 1e-8 1e1 ], 'Color', [ 0.72, 0.27, 1.00 ], 'LineStyle',
        '-.' );
    %
    h5 = line( [ rpm1 rpm1 ], [ 1e-8 1e1 ], 'Color', [ 0.47, 0.67, 0.19 ], 'LineStyle', '-.' );
    h6 = line( [ rpm2 rpm2 ], [ 1e-8 1e1 ], 'Color', [ 0.47, 0.67, 0.19 ], 'LineStyle', '-' );
    %
        h7 = line( [ rpm_plus rpm_plus ], [ 1e-8 1e1 ], 'Color', [ 0.72, 0.27, 1.00 ], 'LineStyle
            ', '-' );
    %
    legend( [ h1 h2 h3 h4 h5 h6 h7 ], 'Washing Machine', 'Raft', 'Load Frequency', 'w-', 'w1', '
        w2', 'w+', 'Location', 'NorthEast' );
    %
    xlabel( 'Rotation [RPM]' );  ylabel( 'Admittance [$\frac{m}{N}$]' );
    axis( [ 6 400  1e-8 1e1 ] );


round( washing_machine.admittance( 3001 ), 3, 'significant' )



%% Problem 2d

dynamic_force.mass = 1;  % kg
dynamic_force.radius = 0.5;  % m


figure( 'Name', 'Displacement' ); ...
    h1 = loglog( rpm, washing_machine.admittance.*h_force( dynamic_force.mass, angular_velocity,
        dynamic_force.radius ).', 'Color', 'r' );  hold on;
    h2 = loglog( rpm, raft.admittance.*h_force( dynamic_force.mass, angular_velocity,
        dynamic_force.radius ).', 'Color', 'b' );
```

```
223     h3 = line( [ 300 300 ], [ 1e-5 1e-1 ], 'Color', 'k', 'LineStyle', '--' );  grid on;
224         text( 220, 0.21, '300 RPM' );
225         legend( [ h1 h2 h3 ], 'Washing Machine', 'Raft', 'Load Frequency', 'Location', 'South' );
226     xlabel( 'Rotation [RPM]' );  ylabel( 'Displacment [m]' );
227     axis( [ 6 400  1e-6 1 ] );
228
229
230 temp = washing_machine.admittance.*h_force( dynamic_force.mass, angular_velocity, dynamic_force.
        radius ).';
231     round( temp( 3001 ), 3, 'significant' )
232
233 % temp = raft.admittance.*h_force( dynamic_force.mass, angular_velocity, dynamic_force.radius )
        .';
234 %     round( temp( 3001 ), 3, 'significant' )
235
236
237
238 %% Problem 2e
239
240 % Displacement of the raft multiplied by the K2 spring constant.
241
242 dynamic_force.mass = 1;  % kg
243 dynamic_force.radius = 0.5;  % m
244
245 h_force = @( force_mass, rotation_speed, load_distance  )  force_mass .* rotation_speed.^2 .*
        load_distance;
246
247 temp = h_force( dynamic_force.mass, angular_velocity, dynamic_force.radius ).';
248
249
250 figure( 'Name', 'Force Applied to Ground by Raft' ); ...
251     h1 = loglog( rpm, raft.admittance.*temp*k2, 'Color', 'b' );  hold on;
252     h2 = line( [ 300 300 ], [ 2e-3 1e-1 ], 'Color', 'k', 'LineStyle', '--' );  grid on;
253     h3 = line( [ 6 400 ], [ 100 100 ], 'Color', 'r' );
254         text( 220, 2e-1, '300 RPM' );
255         legend( [ h1 h2 h3 ], 'Raft Force', 'Load Frequency', 'Maximum Floor Load', 'Location', '
                South' );
256     xlabel( 'Rotation [RPM]' );  ylabel( 'Force Applied to Ground [N]' );
257     axis( [ 6 400  1e-3 1e3 ] );
258
259
260 temp = raft.admittance.*h_force( dynamic_force.mass, angular_velocity, dynamic_force.radius ).'*
        k2;
261     round( temp( 3001 ), 3, 'significant' )
262
263
264
265 %% Problem 2f
266
267 % return
268
269 %% Problem 2g
270
271 % return
272
273 %% Clean-up
274
275 if ( ~isempty( findobj( 'Type', 'figure' ) ) )
276     monitors = get( 0, 'MonitorPositions' );
277         if ( size( monitors, 1 ) == 1 )
278             autoArrangeFigures( 3, 4, 1 );
279         elseif ( 1 < size( monitors, 1 ) )
280             autoArrangeFigures( 2, 2, 2 );
281         end
282 end
283
284 fprintf( 1, '\n\n\n*** Processing Complete ***\n\n\n' );
285
286
287
288 %% Reference(s)
289
290
291
292 %% Comment(s)
293
294
295
```

```
%% Regions of Interest

% 5 regions of interest:

% 1.)  Low frequency:  w < w_minus:
%
%   Stiffness controlled;  two masses moving together;  strong coupling (k3 high).


% 2.)  Force at a resonance frequency:  w = w_minus (lowest resonance frequency):
%
%   At lowest resonance frequency;  at w_minus mode, masses moving in same direction (in-phase)
    with same high amplitude


% 3.) Forcing frequency between w_minus and w2


% 4.)  Forcing frequency w = w2


% 5.)  Forcing frequency w < w < w+
%
%   No a special frequency.


% 6.)  Forcing frequency w = w+
%
% At highest resonance frequency;  at w_plus mode, masses moving in opposite directions (out-of-
    phase) with same high amplitude


% 7.)  Forcing frequency w+ < w
%
% Mass controlled region;  mass 1 (washing machine) moves more than mass 2 (raft);
```

3 Appendix - Matlab Code for Problem 3

```
1
2
3  % To do:
4
5  % remove comment statements
6
7  % verify indices for required values
8
9
10
11 %% Synopsis
12
13 % Problem 3 - Washing Machine Dynamic Vibration Absorber (DVA) Design
14
15
16
17 %% Note(s)
18
19 % At 300 RPM, the load frequency, this system will isolated vibration more
20 % than the 2 DOF and 1 DOF systems.
21
22 % The DVA behaves like a Helmholtz resonator, working as a mechnical notch
23 % filter at a particular frequency.
24
25 % DVAs work well in systems that have one dominate mode.  A good
26 % example of this is building sway and its compensation.  Based on the
27 % design and construction of a building, it will typically have a single,
28 % dominate mode.
29
30 % DVA is useful if you can not get away from the resonance frequency;  can
31 % not tune a 2 DOF system when the resonance frequency is away from the
32 % forcing frequency.
33
34 % DVA introduces two resonances for the original resonance.  These new
35 % resonances will produce greater motion at the new resonance frequencies.
36
37
38
39 %% Environment
40
41 close all; clear; clc;
42 % restoredefaultpath;
43
44 addpath( genpath( './00 Support' ), '-begin' );
45
46 set( groot, 'DefaultFigurePosition', [ 100  750    750  500 ] ); % x, y, width, height
47
48 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
49 set( 0, 'DefaultFigureWindowStyle', 'normal' );
50 set( 0, 'DefaultLineLineWidth', 0.8 );
51 set( 0, 'DefaultTextInterpreter', 'Latex' );
52
53 format ShortG;
54
55 pause( 1 );
56
57
58
59 %% Define Anonymous Functions
60
61 h_rpm_to_f = @( rpm )  ( rpm * 0.10471 ) / ( 2 * pi );
62 h_rpm_to_w = @( rpm )  ( rpm * 0.10471 );
63
64 h_f_to_rpm = @( f )  ( f*2*pi ) / 0.10471;
65
66 h_w_to_f = @( w )  w/(2*pi);
67 h_w_to_rpm = @( w )  h_f_to_rpm( w*2*pi);
68
69
70 h_force = @( force_mass, rotation_speed, load_distance  )  force_mass .* rotation_speed.^2 .*
       load_distance;
71
72
73
74 %% Washing Machine Information
```

```
75
76 machine.mass = 100;  % kg
77 machine.rotation_speed = 31.4;  % radians\s or 300 rotations per minute (RPM)
78 machine.load_mass = 10;  % kg
79 machine.static_displacement = 1e-2;  % m
80
81
82
83 %% Load Force Information
84
85 dynamic_force.mass = 1;  % kg
86 dynamic_force.radius = 0.5;  % m
87
88 % Maximum force applied to foundation is 100 N.
89
90
91
92 %% Dynamic Vibration Absorber (DVA)
93
94 maximum_allowable_displacement = 1e-2;  % m
95     ks = ( machine.load_mass * 9.81) / maximum_allowable_displacement;  % 9,810 N\m
96
97
98 m = 110;  % kg;  washing machine mass and load mass
99 k = ks;  % 9,810 N\m
100
101 wo = sqrt( k / m);  % 9.4 radians\s
102     fo = wo/(2*pi);  % 1.5 Hz
103
104
105 m_dva = 15;  % kg
106 k_dva = 1e5;  % N\m
107
108 wo_dva = sqrt( k_dva / m_dva );  % 81.7 radians\s
109     fo_dva = wo_dva/(2*pi);  % 13.0 Hz
110
111
112
113 %% Blocked Frequencies (Equation 9.1 of the Notes)
114
115 m1 = m_dva;
116 k1 = 0;  % Spring above DVA mass;  does not exist;  set to zero.
117
118 m2 = m;
119 k2 = k;
120
121 k3 = k_dva;
122
123
124 w1 = sqrt( ( k1 + k3 ) / m1 );  % 81.7 radians\s;  smaller than the system resonant frequency.
125     f1 = w1/(2*pi);  % 13 Hz
126
127 w2 = sqrt( ( k2 + k3 ) / m2 );  % 31.6 radians\s;  greater than the system resonant frequency.
128     f2 = w2/(2*pi);  % 5.0 Hz
129
130
131
132 %% Coupled Frequencies (Equation 9.15 of the Notes)
133
134 mu_4 = ( k3^2 / (m1*m2) );  % 6.1 (kg\s)^4
135     mu = ( mu_4 )^0.25;  % 49.1 kg\s
136
137 w(1) = 0.5*( ( w1^2 + w2^2 )  +  sqrt( ( w1^2 - w2^2 )^2 + 4*mu_4 ) );
138 w(2) = 0.5*( ( w1^2 + w2^2 )  -  sqrt( ( w1^2 - w2^2 )^2 + 4*mu_4 ) );
139     w = sqrt( w );
140
141
142 w_minus = w(2);  % 8.9 radians\s  (lower than 31.6 and 81.7 radians\s)
143     f_minus = w(2)/(2*pi);  % 1.4 Hz
144 %
145 w_plus = w(1);  % 87.1 radians\s (greater than 31.6 and 81.7 radians\s)
146     f_plus = w(1)/(2*pi);  % 13.9 Hz
147
148
149 % The blocked frequencies are always between the lower and higher coupled frequencies.
150
151
152
```

```
%% Mode Shapes (Equations 9.18 and 9.19 of the Notes)

phi_plus = [ ...
    1; ...
    -(1/mu^2)*sqrt(m1/m2)*(w_plus^2 - w1^2), ...
    ];
%
%   1
%   -0.14


phi_minus = [ ...
    1; ...
    (1/mu^2)*sqrt(m1/m2)*(w1^2 - w_minus^2), ...
    ];
%
%   1
%   0.99



%% Admittance Using Equations from DVA Lecture

MAXIMUM_RPM = 3e3;

F0 = 1;

w = 0:0.01:h_rpm_to_w( MAXIMUM_RPM );
    f = w/(2*pi);
        rpm = h_f_to_rpm( f );


m = [ 15  110 ];
k = [  0   k_dva.*(1+0.1*1i)   k2.*(1+0.2*1i)   ];


admittance_modified = F0 ./ ( m(1)*m(2) )*k(2) ./ (( w.^2 - w_plus^2 ) .* ( w.^2 - w_minus^2) );
admittance_original = F0 ./ m(2) * 1 ./ ( w2^2 - w.^2 );


figure( 'Name', 'Admittance - Equations from DVA Lecture' ); ...
    h1 = loglog( rpm, abs( admittance_original ) );  hold on;
    h2 = loglog( rpm, abs( admittance_modified ) );
    h3 = line( [ 300 300 ], [ 1e-6 1e-2 ], 'Color', 'k', 'LineStyle', '--' );
        text( 220, 2e-2, '300 RPM' );
    h4 = line( [ h_f_to_rpm( f_minus )  h_f_to_rpm( f_minus ) ], [ 1e-7 1e3 ], 'Color', [ 0.72,
        0.27, 1.00 ], 'LineStyle', '-.' );

    h5 = line( [ h_f_to_rpm( f1 )  h_f_to_rpm( f1 ) ], [ 1e-7 1e3 ], 'Color', [ 0.47, 0.67, 0.19
        ], 'LineStyle', '-.' );
    h6 = line( [ h_f_to_rpm( f2 )  h_f_to_rpm( f2 ) ], [ 1e-7 1e3 ], 'Color', [ 0.47, 0.67, 0.19
        ], 'LineStyle', '--' );

    h7 = line( [ h_f_to_rpm( f_plus )  h_f_to_rpm( f_plus ) ], [ 1e-7 1e3 ], 'Color', [ 0.72,
        0.27, 1.00 ], 'LineStyle', '--' );    grid on;

    axis( [ 5 3e3  1e-7 1 ] );
        legend( [ h1 h2 h3 h4 h5 h6 h7 ], 'Original System', 'DVA System', 'Operating RPM', ...
            '$w_-$', '$w_1$', ...
            '$w_2$', '$w_+$', ...
            'Location', 'NorthWest', 'Interpreter', 'Latex' );
    xlabel( 'Rotation [RPM]' );  ylabel( 'Admittance [$\frac{m}{N}$]' );

return

%% Admittance Using nDOF Function

% Damping can be added by using a dampings vector or appending a complex
% value to the respective spring stiffness.  THESE ARE NOT INTERCHANGABLE.


rpm = 0:0.1:MAXIMUM_RPM;
    f = h_rpm_to_f( rpm );
        w = f ./ (2*pi);

FRF = nDOF_direct_solution( m, k, [ 0 0 0 ], f, 'admittance' );  % Symmetric matrix.
%
squeeze( FRF( numel( f ), :, :) );
```

```
%
% (1, 1) - Force on M1 and its associated displacement.
% (2, 2) - Force on M2 and its associated displacement.
%
% (1, 2) - Force on M1 and the displacement of M2.
% (2, 1) - Force on M2 and the displacment on M1.
%
%   These are interchangeable;  the same result.


figure( 'Name', 'Admittance - nDOF Calculated' ); ...
    h1 = loglog( h_f_to_rpm( f ), abs( FRF( :, 1, 1 ) ), 'Color', 'r' );  hold on;
    h2 = line( [ 300 300 ], [ 1e-6 1e-2 ], 'Color', 'k', 'LineStyle', '--' );
        text( 220, 2e-2, '300 RPM' );
    h3 = line( [ h_f_to_rpm( f_minus )  h_f_to_rpm( f_minus ) ], [ 1e-7 1e3 ], 'Color', [ 0.72,
        0.27, 1.00 ], 'LineStyle', '-.' );

    h4 = line( [ h_f_to_rpm( f1 )  h_f_to_rpm( f1 ) ], [ 1e-7 1e3 ], 'Color', [ 0.47, 0.67, 0.19
        ], 'LineStyle', '-.' );
    h5 = line( [ h_f_to_rpm( f2 )  h_f_to_rpm( f2 ) ], [ 1e-7 1e3 ], 'Color', [ 0.47, 0.67, 0.19
        ], 'LineStyle', '--' );

    h6 = line( [ h_f_to_rpm( f_plus )  h_f_to_rpm( f_plus ) ], [ 1e-7 1e3 ], 'Color', [ 0.72,
        0.27, 1.00 ], 'LineStyle', '--' );    grid on;

    axis( [ 2e1 MAXIMUM_RPM  1e-7 1 ] );
        legend( [ h1 h2 h3 h4 h5 h6 ], 'DVA System', 'Operating RPM', ...
            '$w_-$', '$w_1$', ...
            '$w_2$', '$w_+$', ...
            'Location', 'NorthWest', 'Interpreter', 'Latex' );
    xlabel( 'Rotation [RPM]' );  ylabel( 'Admittance [$\frac{m}{N}$]' );


% temp2 = abs( FRF( :, 1, 1 ) );
    % round( temp2( 3001 ), 3, 'significant' )



%% Displacement

dynamic_force.mass = 1;  % kg
dynamic_force.radius = 0.5;  % m


figure( 'Name', 'Displacement' ); ...
    h1 = loglog( rpm, abs( FRF( :, 1, 1 ) ).*h_force( dynamic_force.mass, w, dynamic_force.radius
        ).', 'Color', 'r' );  hold on;
    h2 = line( [ 300 300 ], [ 1e-7 1e-3 ], 'Color', 'k', 'LineStyle', '--' );  grid on;
        text( 220, 2e-3, '300 RPM' );
    h4 = line( [ h_f_to_rpm( f_minus )  h_f_to_rpm( f_minus ) ], [ 1e-8 1e3 ], 'Color', [ 0.72,
        0.27, 1.00 ], 'LineStyle', '-.' );

    h5 = line( [ h_f_to_rpm( f1 )  h_f_to_rpm( f1 ) ], [ 1e-8 1e3 ], 'Color', [ 0.47, 0.67, 0.19
        ], 'LineStyle', '-.' );
    h6 = line( [ h_f_to_rpm( f2 )  h_f_to_rpm( f2 ) ], [ 1e-8 1e3 ], 'Color', [ 0.47, 0.67, 0.19
        ], 'LineStyle', '--' );

    h7 = line( [ h_f_to_rpm( f_plus )  h_f_to_rpm( f_plus ) ], [ 1e-8 1e3 ], 'Color', [ 0.72,
        0.27, 1.00 ], 'LineStyle', '--' );    grid on;

        legend( [ h1 h2 h3 h4 h5 h6 ], 'DVA System', 'Operating RPM', ...
            '$w_-$', '$w_1$', ...
            '$w_2$', '$w_+$', ...
            'Location', 'NorthWest', 'Interpreter', 'Latex' );

    xlabel( 'Rotation [RPM]' );  ylabel( 'Displacment [m]' );
    axis( [ 2e1 MAXIMUM_RPM  1e-8 1 ] );


% temp2 = abs( FRF( :, 1, 1 ) ).*h_force( dynamic_force.mass, w, dynamic_force.radius ).';
    % round( temp2( 3001 ), 3, 'significant' )



%% Ground Force

dynamic_force.mass = 1;  % kg
dynamic_force.radius = 0.5;  % m
```

```
296
297
298 figure( 'Name', 'Ground Force' ); ...
299     h1 = loglog( rpm, abs( FRF( :, 1, 1 ) ).*h_force( dynamic_force.mass, w, dynamic_force.radius
          ).'.*k2, 'Color', 'k' );  hold on;
300     h2 = line( [ 300 300 ], [ 1e-4 1e-0 ], 'Color', 'k', 'LineStyle', '--' );  grid on;
301         text( 220, 2e-0, '300 RPM' );
302     h3 = line( [ 6 MAXIMUM_RPM ], [ 100 100 ], 'Color', 'r' );
303         legend( [ h1 h2 h3 ], 'Ground Force', 'Operating RPM', 'Maximum Floor Load', 'Location',
              'SouthEast' );
304     xlabel( 'Rotation [RPM]' );  ylabel( ' Force [N]' );
305     axis( [ 2e1 MAXIMUM_RPM  1e-5 1e3 ] );
306
307
308 temp2 = abs( FRF( :, 1, 1 ) ).*h_force( dynamic_force.mass, w, dynamic_force.radius ).'*k2;
309     % round( temp2( 3001 ), 3, 'significant' )
310
311
312
313 %% Clean-up
314
315 if ( ~isempty( findobj( 'Type', 'figure' ) ) )
316     monitors = get( 0, 'MonitorPositions' );
317         if ( size( monitors, 1 ) == 1 )
318             autoArrangeFigures( 3, 4, 1 );
319         elseif ( 1 < size( monitors, 1 ) )
320             autoArrangeFigures( 2, 2, 2 );
321         end
322 end
323
324 fprintf( 1, '\n\n\n*** Processing Complete ***\n\n\n' );
325
326
327
328 %% Reference(s)
```

4 Appendix - Matlab Code for Problem 4

```
1
2
3
4  %% Synopsis
5
6  % Problem 4 - Washing Machine Abuse Testing
7
8
9
10 %% Environment
11
12 close all; clear; clc;
13 % restoredefaultpath;
14
15 addpath( genpath( './00 Support' ), '-begin' );
16
17 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
18 set( 0, 'DefaultFigureWindowStyle', 'normal' );
19 set( 0, 'DefaultLineLineWidth', 0.8 );
20 set( 0, 'DefaultTextInterpreter', 'Latex' );
21
22 format ShortG;
23
24 pause( 1 );
25
26
27
28 %% Impulse Signal
29
30 time_step = 1e-4;  % s
31 net_time = 10;  % s
32     time_indices = 0:time_step:( net_time - time_step );
33
34 impulse = zeros( size( time_indices ) );
35     impulse( 4/time_step ) = 1./time_step;
36         brick_impulse = 5 .* impulse;
37         % brick_impulse = 50 .* impulse;
38
39
40 % figure( 'Name', 'Unit Impulse Signal' ); ...  % Figure 1
41 %     plot( time_indices, impulse );  grid on;
42 %         legend( 'Unit Impulse Signal', 'Location', 'NorthEast' );
43 %     xlabel( 'Time [s]' );  ylabel( 'Force [N]' );
44 %     axis( [ 0 net_time  0 125 ] );
45
46 % return
47
48 %% 1 DOF System Impulse Function
49
50 m = 5;  % kg
51
52 % Initial conditions of 1 DOF system.
53 xo = 0;  % m
54 vo = 1 / m;  % m\s
55
56 wd = 31.4;  % radians\s or 5 Hz or 300 RPM
57
58
59 ks = 9810;  % N\m
60     wo = sqrt( ks / 110 );  % 9.4 radians\s
61         fo = wo / (2*pi);  % 1.5 Hz
62
63
64 epsilon = 0.25;
65     C = epsilon * 2*sqrt( ks * 110 );  % 519.4 kg\s
66
67
68 h_unit_impulse = @( washing_machine_mass, wd, wo, epsilon, t )  ( 1./( washing_machine_mass.*wd)
       )  .*  exp( -wo.*epsilon.*t ) .* sin( wd.*t );  % mvo = 1 kg\(m s)
69
70
71
72 %% 1 DOF System Impulse Response
73
74 impulse_response = h_unit_impulse( 100, wd, wo, epsilon, time_indices );
```

```
%
figure( 'Name', '1 DOF System Impulse Response' ); ... % Figure 2
    plot( time_indices, impulse_response );  grid on;
    xlabel( 'Time [s]' );  ylabel( 'Admittance [$\frac{m}{N}$]' );
    axis( [ -0.1 3  -2.5e-4 3.0e-4 ] );


response_to_brick_perturbation = conv( brick_impulse, impulse_response ) * time_step;
    time_indices_perturbation = ( 0:1:( numel( response_to_brick_perturbation ) - 1 ) ) .*
        time_step;
%
figure( 'Name', '1 DOF Impulse Response' ); ... % Figure 3
    plot( time_indices_perturbation, response_to_brick_perturbation );  grid on;
    xlabel( 'Time [s]' );  ylabel( 'Displacement [m]' );
    axis( [ -0.1 5  -1.5e-3 1.5e-3 ] );



%% Sinusoidal Impulse Response

% sinusoid_input = 493.5 * sin( 2 * pi * 5 * time_indices );
sinusoid_input = 1 * sin( 2 * pi * 5 * time_indices );
% sinusoid_input = 493.5 * sin( 2 * pi * 1 * time_indices );
    sinusoid_input( 1:1:( 1 / time_step ) ) = 0;
    sinusoid_input( ( 8 / time_step ):1:( 10 / time_step ) ) = 0;
%
figure( 'Name', 'Sinusoidal Input' ); ... % Figure 4
    plot( time_indices, sinusoid_input );  grid on;
    xlabel( 'Time [s]' );  ylabel( 'Force [N]' );
    % xlim( [ 0 2.5 ] );
    ylim( [ -600 600 ] );


response_to_sinusoidal_forcing = conv( sinusoid_input, impulse_response ) * time_step;
    time_indices_sinusoidal_forcing = ( 0:1:( numel( response_to_sinusoidal_forcing ) - 1 ) ) .*
        time_step;
%
figure( 'Name', '1 DOF Sinusoidal Forcing Response' ); ... % Figure 5
    plot( time_indices_sinusoidal_forcing, response_to_sinusoidal_forcing );  grid on;
    xlabel( 'Time [s]' );  ylabel( 'Displacement [m]' );
    % xlim( [ 0 2.5 ] );



%% Combined Response

joint_signal = sinusoid_input + brick_impulse;
%
figure( 'Name', 'Combined Signal' ); ... % Figure 6
    plot( time_indices, joint_signal );  hold on;
    plot( time_indices, sinusoid_input, 'LineStyle', '--' );
    plot( time_indices, impulse, 'LineStyle', '--' );
    xlabel( 'Time [s]' );  ylabel( 'Force [N]' );
    % xlim( [ 0 2.5 ] );
    % ylim( [ -600 600 ] );


response_to_combined_forcing = conv( impulse_response, joint_signal ) .* time_step;
    time_indices_sinusoidal_forcing = ( 0:1:( numel( response_to_brick_perturbation ) - 1 ) ) .*
        time_step;
%
figure( 'Name', '1 DOF Combined Forcing Response' ); ... % Figure 7
    plot( time_indices_sinusoidal_forcing, response_to_combined_forcing );  grid on;
    xlabel( 'Time [s]' );  ylabel( 'Displacement [m]' );
    % xlim( [ 0 2.5 ] );



%% Plot Difference

figure( 'Name', '1 DOF Combined Forcing Response' ); ... % Figure 7
    plot( time_indices_sinusoidal_forcing, response_to_combined_forcing -
        response_to_sinusoidal_forcing );  grid on;
    xlabel( 'Time [s]' );  ylabel( 'Displacement [m]' );
    % xlim( [ 0 2.5 ] );



```

```
%% Clean-up

if ( ~isempty( findobj( 'Type', 'figure' ) ) )
    monitors = get( 0, 'MonitorPositions' );
        if ( size( monitors, 1 ) == 1 )
            autoArrangeFigures( 3, 4, 1 );
        elseif ( 1 < size( monitors, 1 ) )
            autoArrangeFigures( 2, 2, 2 );
        end
end

fprintf( 1, '\n\n\n*** Processing Complete ***\n\n\n' );


%% Reference(s)
```